

Strings

AP Computer Science A
Unit 1B - Using Objects

Character Strings

- Every character string is an object in Java, defined by the String class.
- Every string literal, delimited by double quotation marks, represents a String object.
- The String concatenation operator(+) is used to append one String to the end of another.
  - It can also be used to append a number to a String.
- A string literal cannot be broken across two lines in a program.

Example

```
public class Facts
{
	public static void main(String [ ] args)
	{
		// Strings can be concatenated into one long string
		System.out.println("We present the following facts for your "
			+ "extracurricular edification: ");
		System.out.println();

		// A string can contain numeric digits
		System.out.println("Letters in the Hawaiian alphabet:  12");

		// A numeric value can be concatenated to a string
		System.out.println("Dialing code for Antarctica:  " + 672);

		System.out.println("Year in which Leonardo da Vinci invented "
			+ "the parachute:  " + 1515);

		System.out.println("Speed of ketchup:  " + 40 + " km per year");
	}
}
```

Output:

```
We present the following facts for your extracurricular
edification:

Letters in the Hawaiian alphabet:  12
Dialing code for Antarctica:  672
Year in which Leonardo da Vinci invented the parachute:  1515
Speed of ketchup:  40 km per year
```

String Concatenation

- The plus operator(+) is also used for arithmetic addition.
- The function that the + operator performs depends on the type of the information on which it operates.
    - If both operands are Strings, or if one is a String and one is a number, it performs string concatenation.
    - If both operands are numeric, it adds them.
- The + operator is evaluated left to right.
- Parentheses can be used to force the operation order.

Example

```
public class Addition
{
    public static void main(String [ ] args)
    {
        System.out.println("21 and 43 concatenated:  " + 21 + 43);
        System.out.println("21 and 43 added:  " + (21 + 43));
        System.out.println(21 + 43 + " looks like this");
        System.out.println("" + 21 + 43 + " looks like this");
    }
}
```

Output:

```
21 and 43 concatenated:  2143
24 and 43 added:  64
64 looks like this
2143 looks like this
```

Creating Objects

- The keyword new is used to create all objects.
  - `String curriculum = new String("CS Awesome");`
- Because Strings are so common, we don't have to use the new operator to create a String object.
  - `String title = "CSAwesome2";`
- This is special syntax that works only for Strings and its referred to as a string literal.
- Once an object has been instantiated, we can use the dot operator to invoke its methods.
  - `title.length();`

What is a String?

- A string holds characters in a sequence.
- Each character is at a set position or index which starts with 0 as shown below.
- An index is a number associated with a position in a String.
- The length of a String is the number of characters in it, including any spaces or special characters.
- The String below has a length of 14.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
String	T	h	i	s		i	s		a		t	e	s	t

What is a String Method?

- The String class is in the java.lang package.
  - Classes in the java.lang package are available by default.
- The String class has many methods. You are only required to be able to use a few of them that are specified in the APCS A Java subset.
- You can learn more about the String class by viewing the Applications Program Interfaces(APIs), often referred to as the Java Docs.
  - https://docs.oracle.com/javase/7/docs/api/java/lang/String.html
- Many of the methods return a value, such as an int or a new String object.

Quick Reference Guide String Methods

Class Constructors and Methods	Explanation
String Class	
`String(String str)`	Constructs a new `String` object that represents the same sequence of characters as `str`
`int length()`	Returns the number of characters in a `String` object
`String substring(int from, int to)`	Returns the substring beginning at index `from` and ending at index `to – 1`
`String substring(int from)`	Returns `substring(from, length())`
`int indexOf(String str)`	Returns the index of the first occurrence of `str`; returns `-1` if not found
`boolean equals(Object other)`	Returns `true` if `this` corresponds to the same sequence of characters as `other`; returns `false` otherwise
`int compareTo(String other)`	Returns a value < 0 if `this` is less than `other`; returns zero if `this` is equal to `other`; returns a value > 0 if `this` is greater than `other`. Strings are ordered based upon the alphabet.
`String[] split(String del)`	Returns a `String` array where each element is a substring of `this String`, which has been split around matches of the given expression `del`

String Methods

- **`String`**`(String str)` - Constructor: creates a new String object with the same characters as str.
  - `String name1 = "This is a test";`
  - `String name2 = new String("This is a test");`

String Methods

- `int length()` - Returns the number of characters in this String.
- `int indexOf(String str)` - Returns the position of the first character in the first occurrence of str in this String. Returns -1 if str is not found.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	length = 14
String	T	h	i	s		i	s		a		t	e	s	t	

Strings

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	length = 14
String	T	h	i	s		i	s		a		t	e	s	t	

- `String s = "This is a test";`
- `s.length()`
  - indices start at 0 and go to `s.length()-1`
- `s.indexOf(otherStr)`
  - returns the index of the first occurrence of `otherStr`
  - returns -1 if `otherStr` not found

String Practice

```
My code works... I have no idea why.
012345678901234567890123456789012345
```

Given: `String phrase = "My code works... I have no idea why."`, evaluate each of the following.

- `phrase.length();`
- `phrase.indexOf("I");`
- `phrase.indexOf("d");`
- `phrase.indexOf("no");`
- `phrase.indexOf("A");`

String Methods

- `String substring(int from, int to)` - Returns a new String that is a subset of this String starting at index from and ending with the character at position(to - 1).
- `String substring(int from)` - Returns a new String that starts at index from and extends to the end of the String.

Substring

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	length = 14
String	T	h	i	s		i	s		a		t	e	s	t	

- `s.substring(start,end)`
  - does not include the character at the `end` index (Not inclusive)
  - Length of substring is `end` - `start`
  - `s.substring(i,i+1)` returns the character at index i
  - "Draw a wall" in front of `start` and in front of `end`, that's your substring

Substring

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	length = 14
String	T	h	i	s		i	s		a		t	e	s	t	

`s.substring(from)`

- Gives all characters at and after from (including from)
- Same as `s.substring(front, s.length())`
  - no error because it doesn't include `s.length()`
- "Draw a wall" in front of `start` and take everything to the right, that's your substring

Substring

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	length = 14
String	T	h	i	s		i	s		a		t	e	s	t	

- `s.substring(3,4)`
- `s.substring(6,6)`
- `s.substring(10,13)`
- `s.substring(10,14)`
- `s.substring(10,15)`

"Draw a wall" in front of `from` and in front of `to`, that's your substring

String Practice

```
My code works... I have no idea why.
012345678901234567890123456789012345
```

Given: `String phrase = "My code works... I have no idea why."`, evaluate each of the following.

- `phrase.substring(4);`
- `phrase.substring(4, 10);`
- `phrase.substring(12);`
- `phrase.substring(0, 3) + phrase.substring(20);`

String Practice

```
String str = new String("Computer science students rock!");
```

1. Write the Java statement that would return the string `"science students"` from str using the substring method.
2. What string would be returned by the statement
   ```
   String result = str.substring(3,9);
   ```
3. Write the two Java statements that would return `"students rock!"` from `str` using the 2 substring methods.

Comparing Strings

- Remember that in Java, a String is an object.
- We cannot use the relational operators to compare Strings.
  - == checks if the variables point to the same reference
  - != checks if they are the different references
  - <, <=, >, >= do not do anything

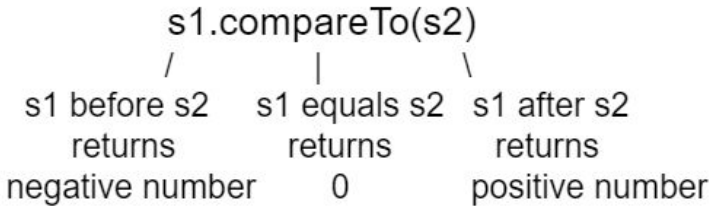
Comparing Strings for Equality

```
String s1 = "hello";
String s2 = new String("hello");
String s3 = "hello";
String s4 = s1;
```

- `s1 == s2` → `false`
- `s1 == s3` → `probably true bc of Java Strings`
- `s1 == s4` → `true because they are aliases`

Strings

- `s1.equals(s2)` - returns true if they are the same, false if not
- `s1.compareTo(s2)`
  - returns a number < 0 if s1 < s2 (alphabetically)
  - returns a number > 0 if s1 > s2
  - returns 0 if s1 == s2



String Methods

- `int compareTo(String other)` - Returns a number indicating whether this String comes before(a negative return value), is equal to(a zero return value), or comes after(a positive return value) the String other.
- `boolean equals(Object other)` - Returns true if this String contains the same characters as other(including upper or lowercase) and false if it does not.

Comparing Strings for Equality

```
String first = "ace";
String second = "aces";
```

- Would the following be true or false?
  - `first.equals(second);`
  - `"ace".equals(first);`
  - `first.equals("ACE");`
  - `second.equals(first + "s");`
  - `first.equals(second.substring(0, 3));`

Lexicographic Ordering

- Because comparing characters and Strings is based on a character set, it is called lexicographic ordering.
  - This is not strictly alphabetical when uppercase and lowercase characters are mixed.
  - For example, the String "Great" comes before the String "fantastic" because uppercase letters come before lowercase letters in Unicode.
- Also, short Strings come before longer Strings with the same prefix.
  - Therefore, "book" comes before "bookcase".

compareTo()

- `compareTo()` returns a negative, zero, or positive value if the String is smaller, equal, or larger than the argument, respectively.

```
String test = "XYZ";

"XYZ".compareTo("ACB");     // returns a positive value:  'X' > 'A'

test.compareTo("XYZ");      // returns zero:  "XYZ" equals "XYZ"

test.compareTo("xyz");      // returns a negative value:  'X' < 'x'

"acts".compareTo("aces");   // returns a positive value:  't' > 'e'
```